Final Capstone Project Report – BookMyShow CI/CD & Deployment

Submitted by:

Bimal Kumar Behera

Batch 1

Project Goal

Automating the CI/CD pipeline for BookMyShow application with GitHub, Jenkins, Docker, Kubernetes (EKS), and Monitoring (Prometheus & Grafana).

Tools & technologies used

Jira, GitHub, Jenkins, SonarQube, Docker, DockerHub, Kubernetes (EKS), Prometheus, Grafana

JIRA

- A Jira board was created with a simple workflow: To Do → In Progress → Done to track project tasks.
- Assigned all tasks to myself to efficiently monitor progress and manage work.

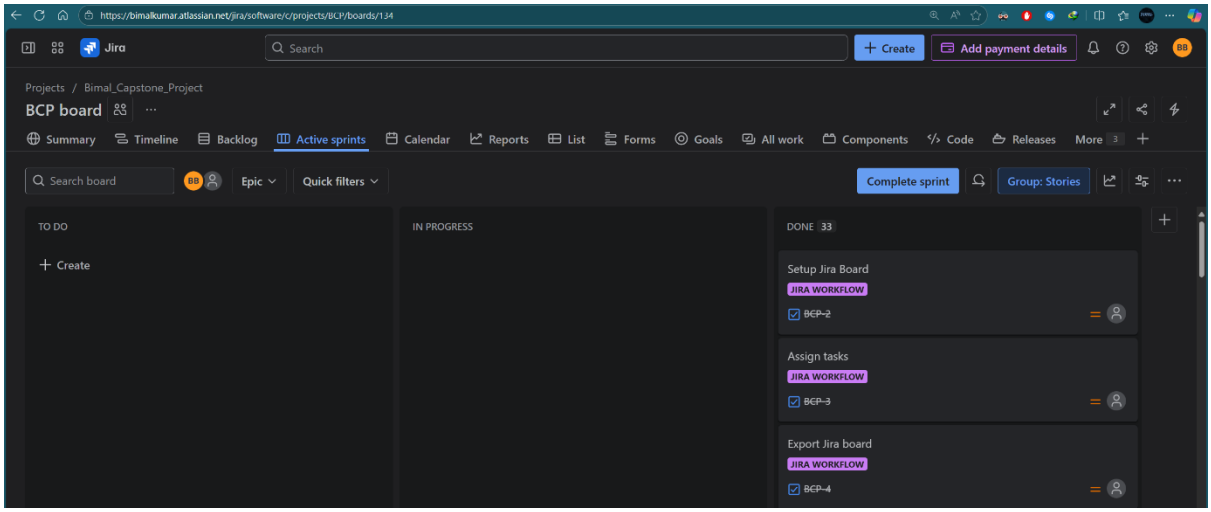


Figure 1: Jira board showing Jira Workflow tasks completed

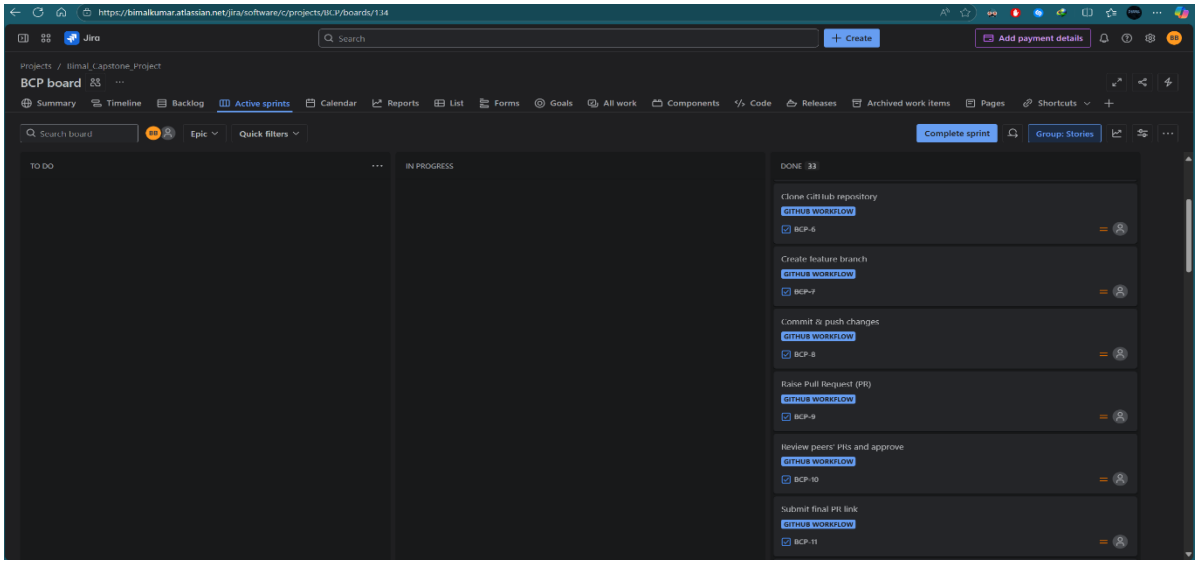


Figure 2: Jira board showing GitHub Workflow tasks completed

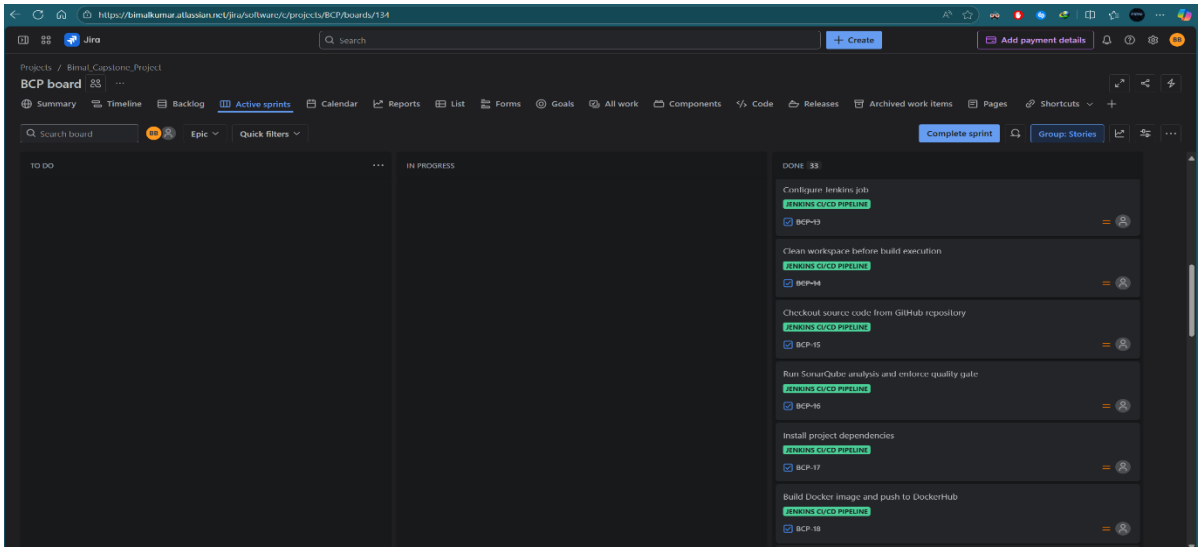


Figure 3: Jira board showing Jenkins CI/CD Pipeline tasks completed

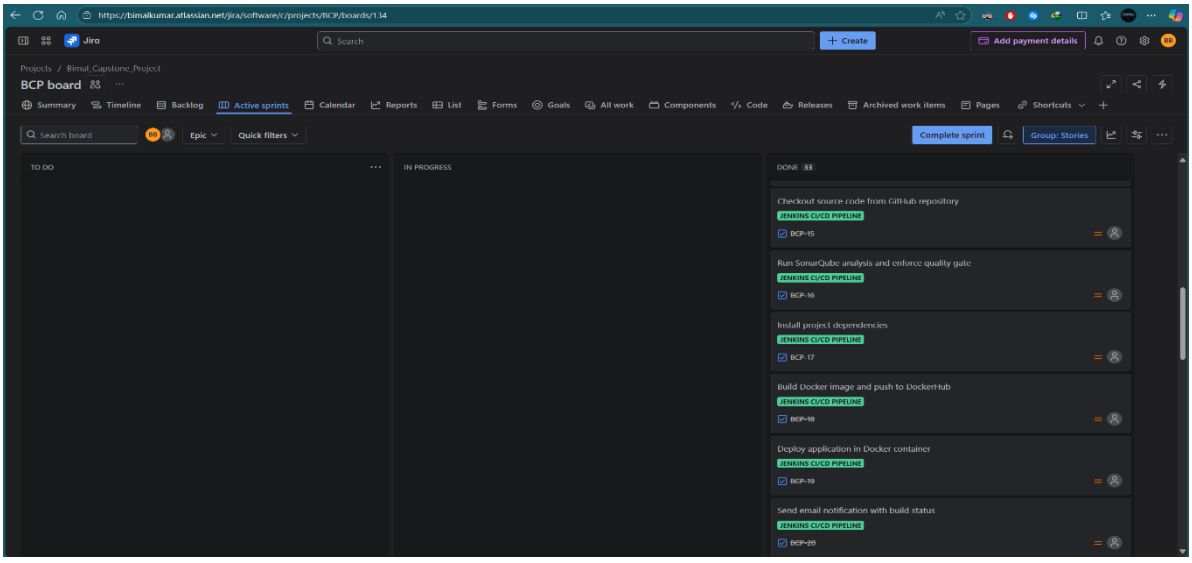


Figure 4: Jira board showing Jenkins CI/CD Pipeline tasks completed

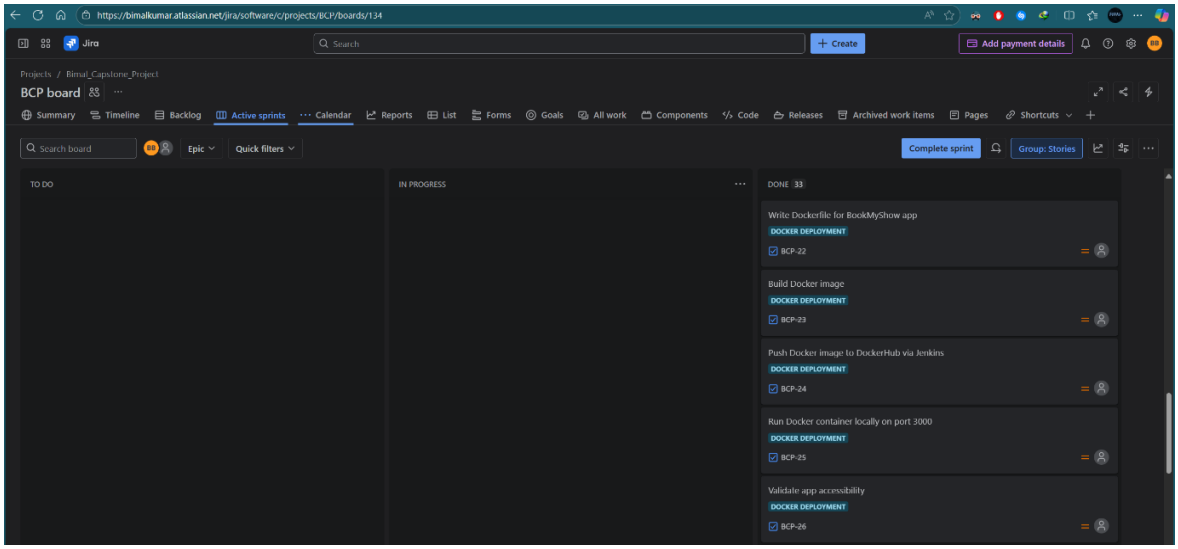


Figure 5: Jira board showing Docker tasks completed

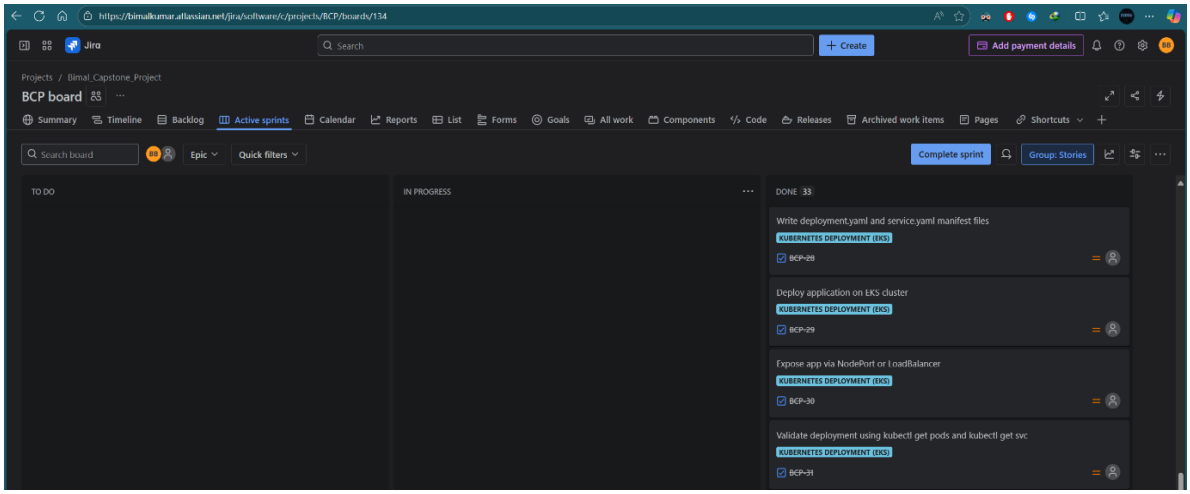


Figure 6: Jira board showing Kubernetes tasks completed

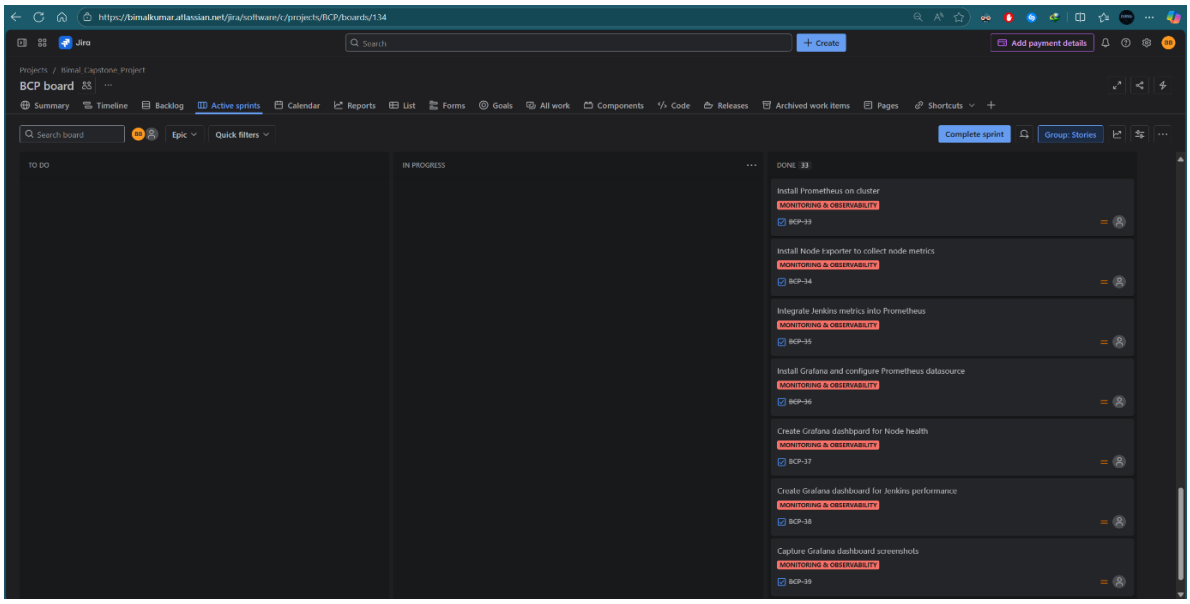


Figure 7: Jira board showing Monitoring & Observability tasks completed

GitHub

- First, I forked the repository and then cloned the GitHub repository and worked on a feature branch.
- Changes were pushed via a Pull Request (PR), reviewed, and merged into the main branch.

GitHub link: https://github.com/bimalk07/Book-My-Show.git

Pull Request (PR) links:

https://github.com/bimalk07/Book-My-Show/pull/1/

https://github.com/bimalk07/Book-My-Show/pull/2/

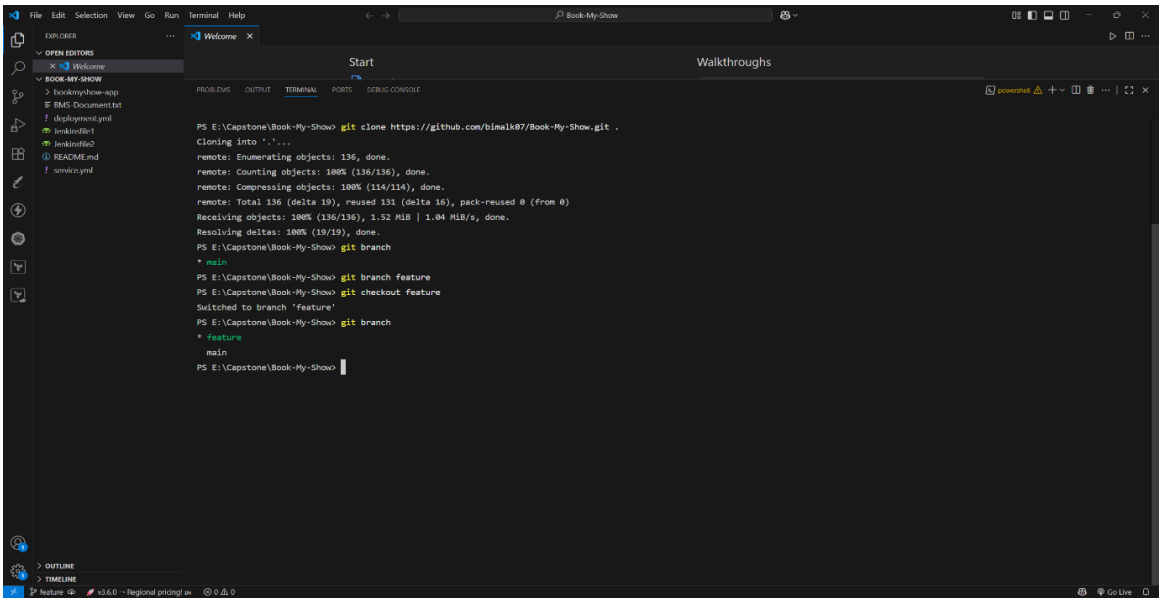


Figure 8: Repository cloned and feature branch created

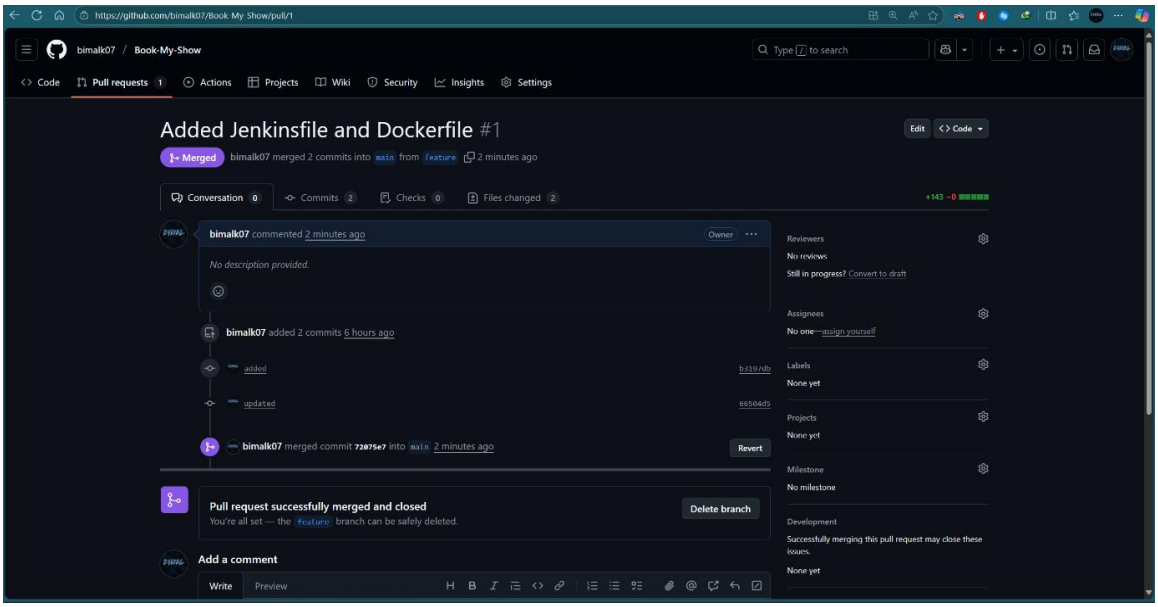


Figure 9: Pull Request #1 created and submitted for merging

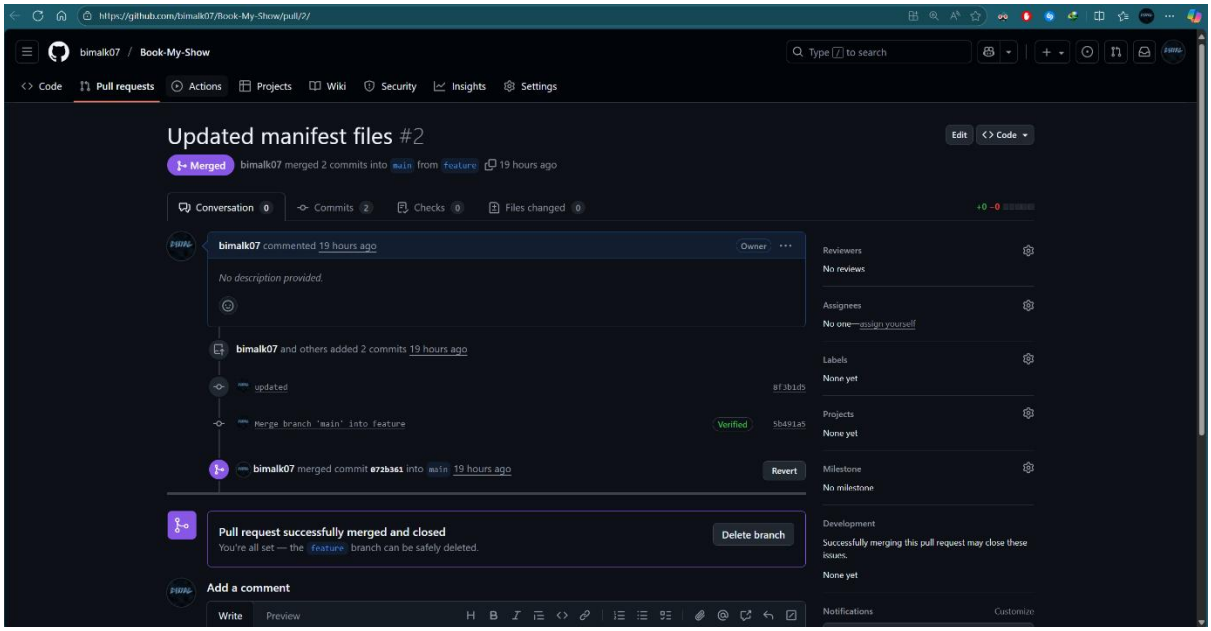


Figure 10: Pull Request #2 created and submitted for merging

Jenkins CI/CD Pipeline

EC2 Instance Setup:

- Created two Ubuntu EC2 instances: one for Jenkins and the other for running SonarQube as a Docker container.
- Installed Java, Jenkins, and Docker for building, deploying, and managing applications.
- Deployed SonarQube as a Docker container on the second server to enable code quality analysis.

Plugins used:

- Eclipse Termurin installer (for JDK support)
- Pipeline stage view
- SonarQube scanner
- Sonar Quality Gates
- Docker
- Docker commons
- Docker pipeline
- Docker API
- Docker build step

Credentials configuration:

- Added SonarQube token, DockerHub personal access token to Jenkins credentials for secure authentication.

CI/CD Pipeline:

- Created a Jenkins pipeline with multiple stages including
  - Cleaning the workspace
  - Checking out code from GitHub
  - Running SonarQube analysis
  - Installing dependencies
  - Building & pushing Docker images
  - Deploying containers
  - Email notifications for build status.

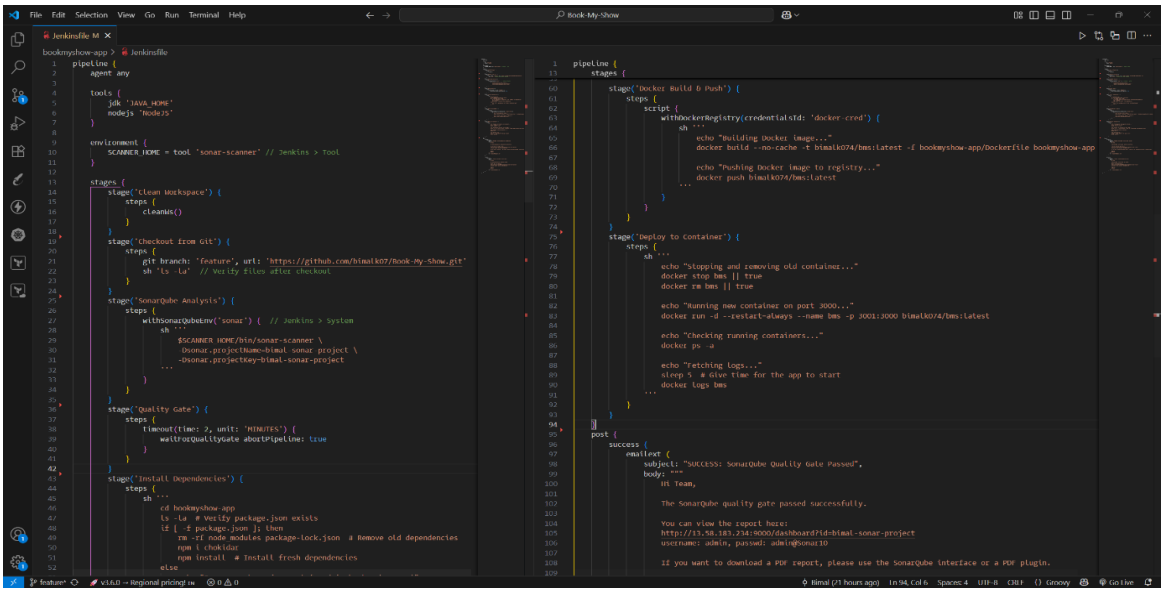


Figure 11: Jenkinsfile

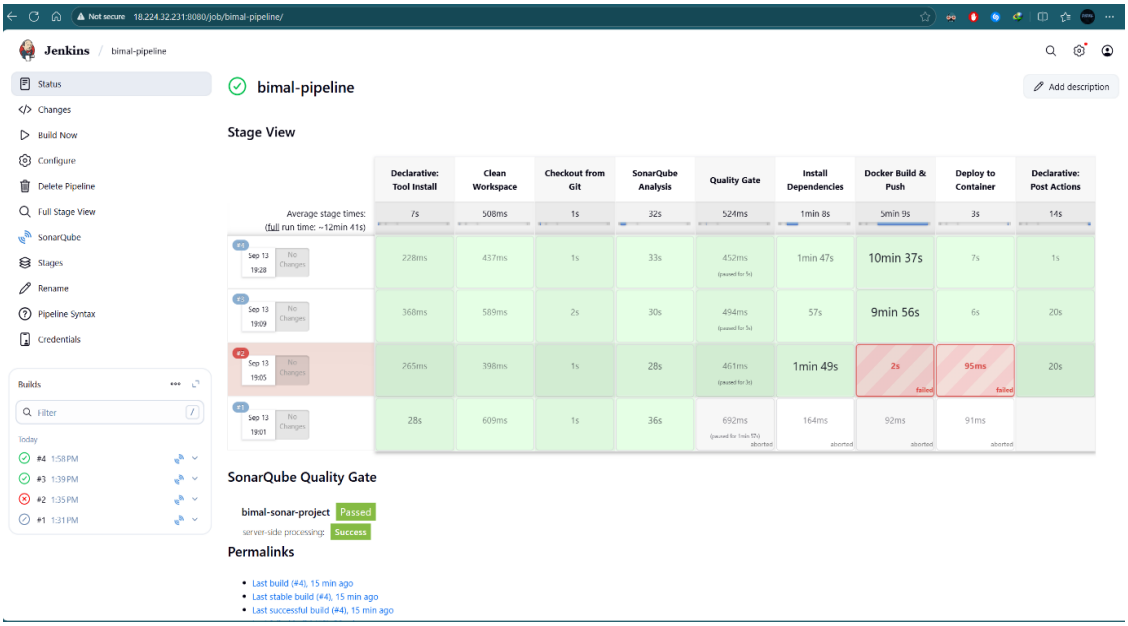


Figure 12: Jenkins pipeline stage view showing successful execution of all stages

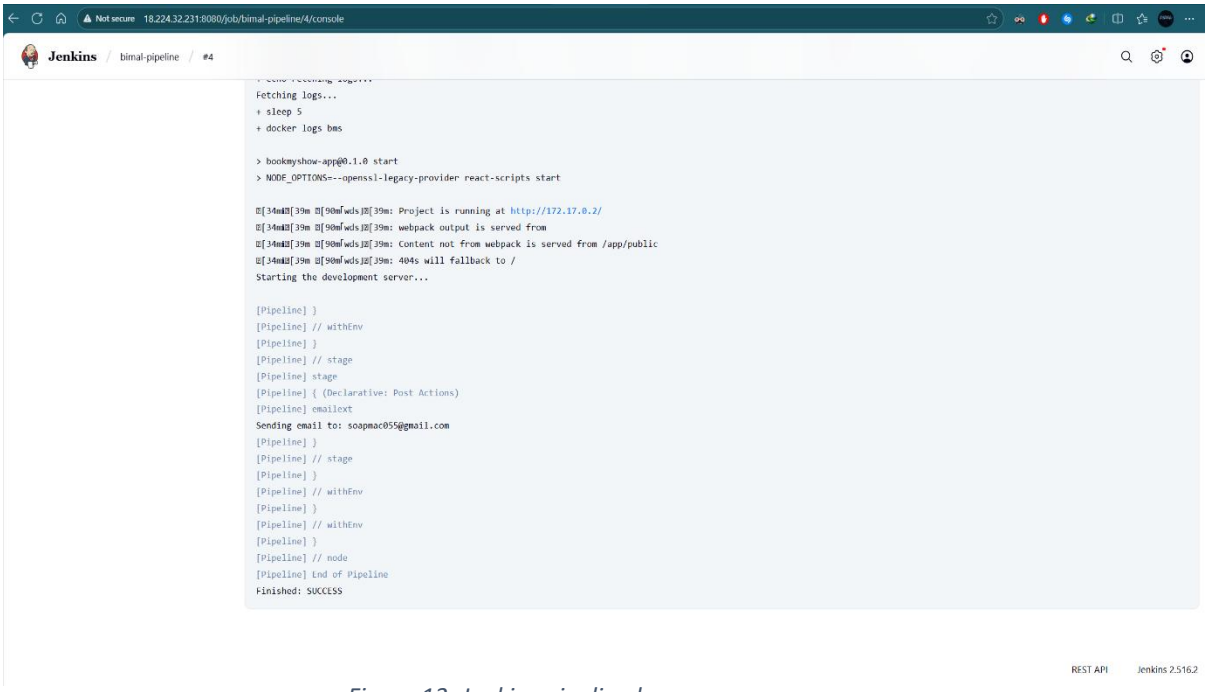


Figure 13: Jenkins pipeline logs success message

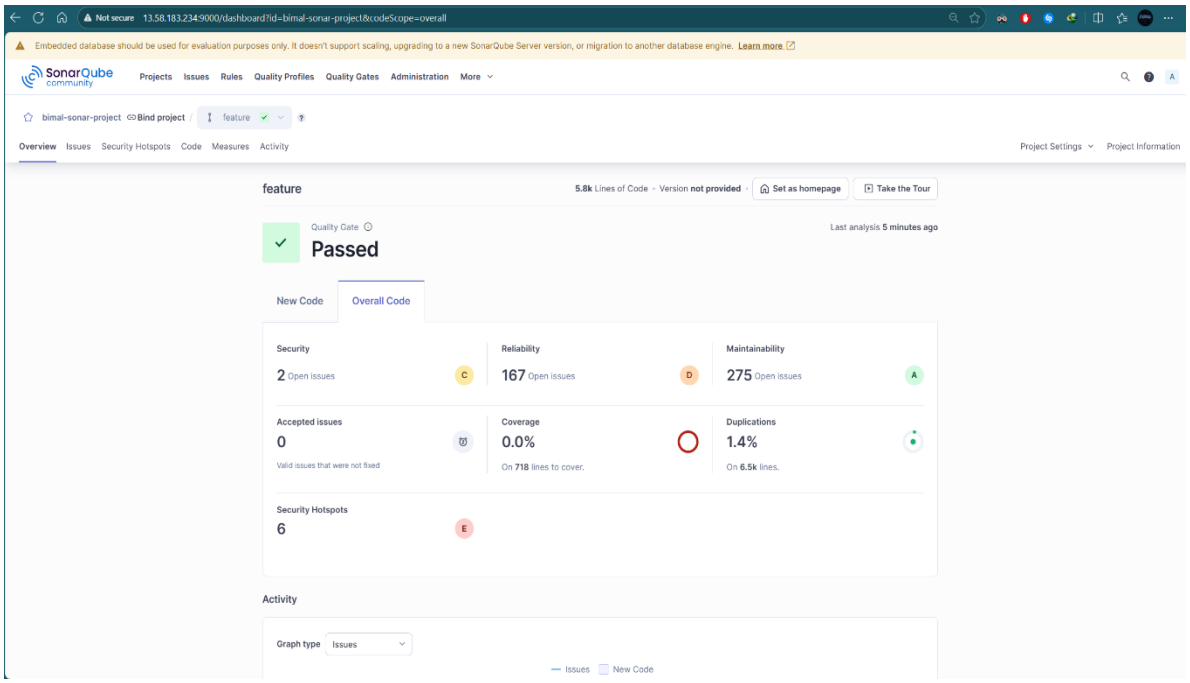


Figure 14: SonarQube dashboard showing overall code analysis passed

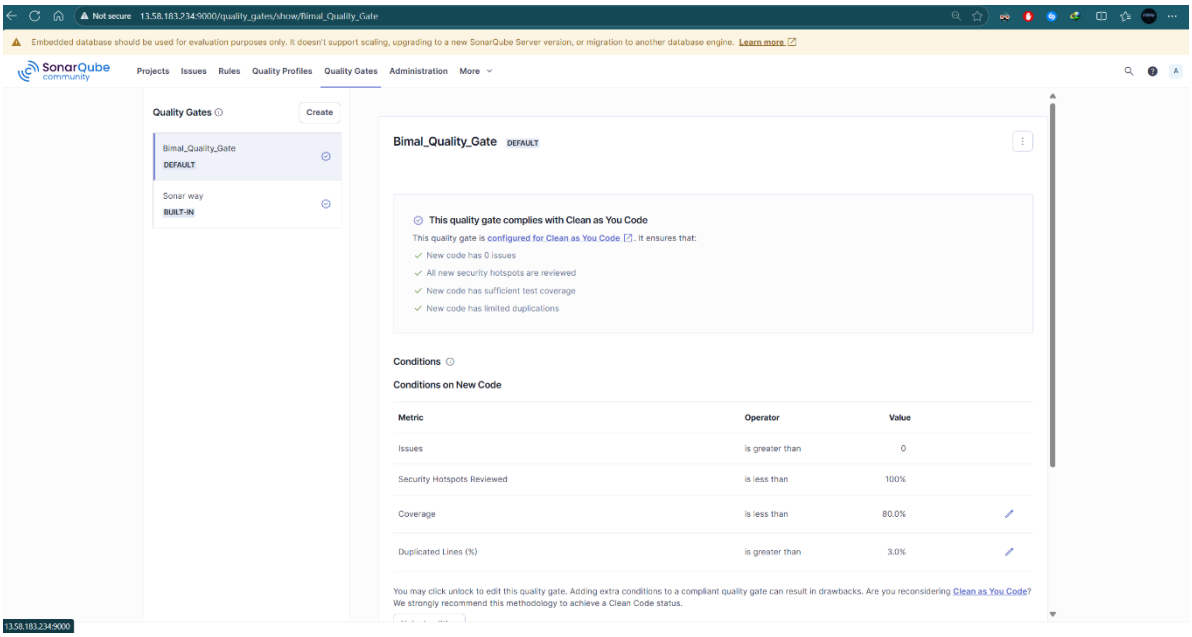


Figure 15: SonarQube Quality Gate

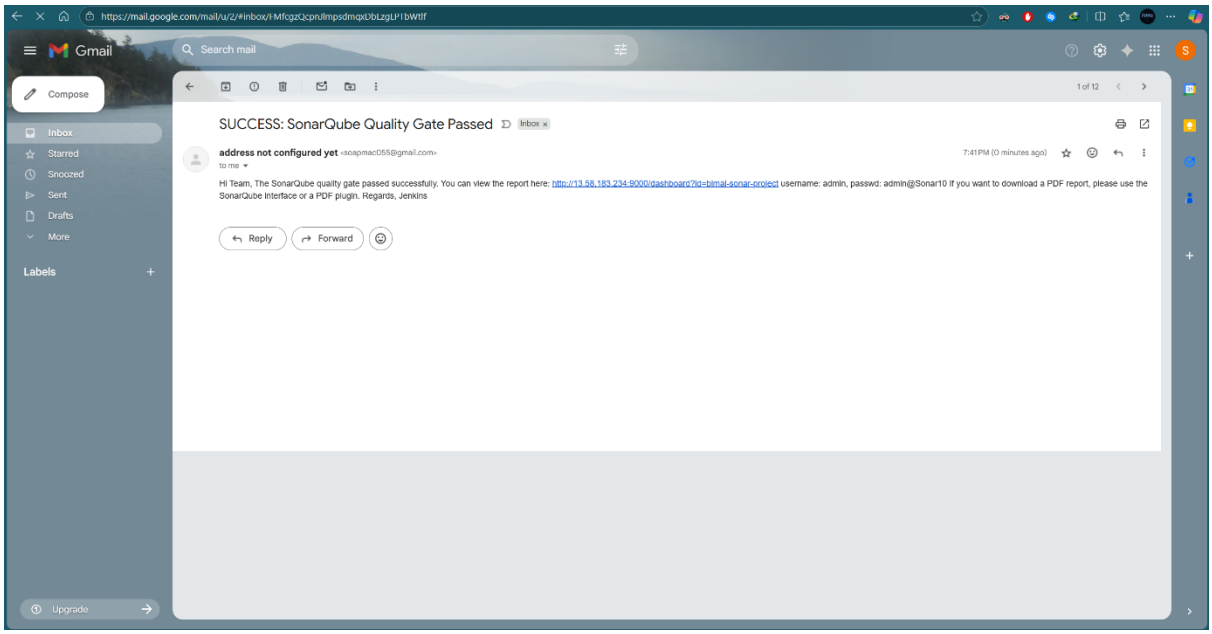


Figure 16: Email notification received for successful Jenkins build

Docker Deployment

- A Dockerfile was created to containerize the BookMyShow application.
- The Docker image was built, pushed to DockerHub, and validated by running a container locally on port 3000 to confirm application accessibility.

Docker image link:
https://hub.docker.com/repository/docker/bimalk074/bms/tags

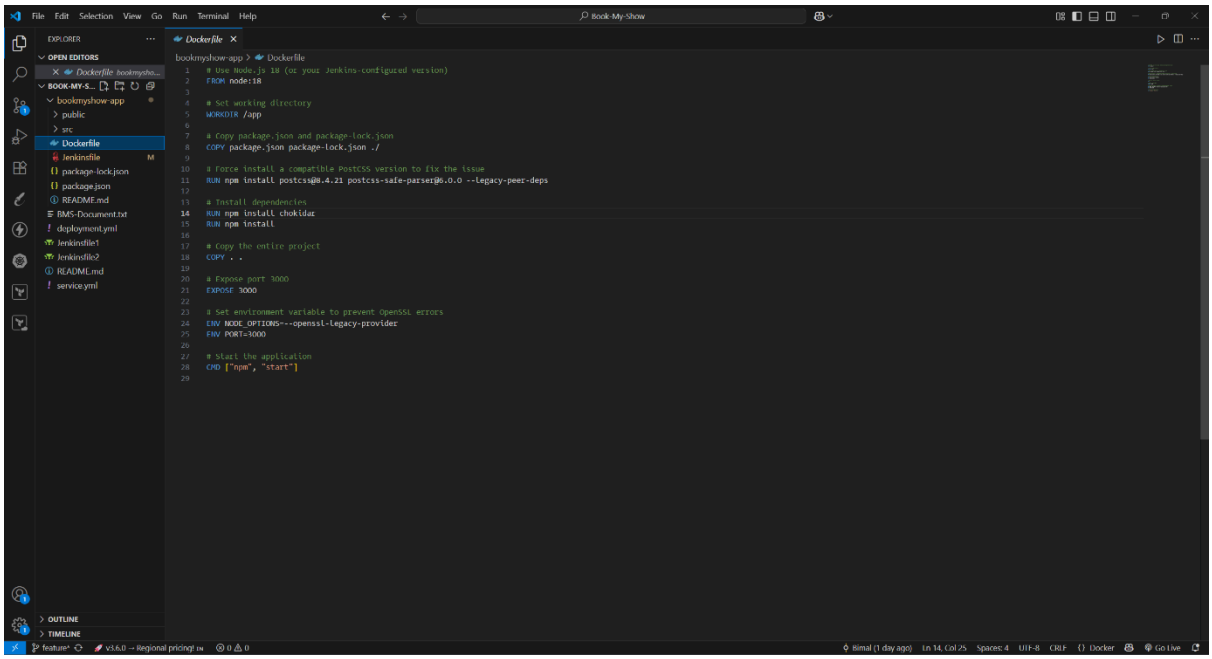


Figure 17: Dockerfile

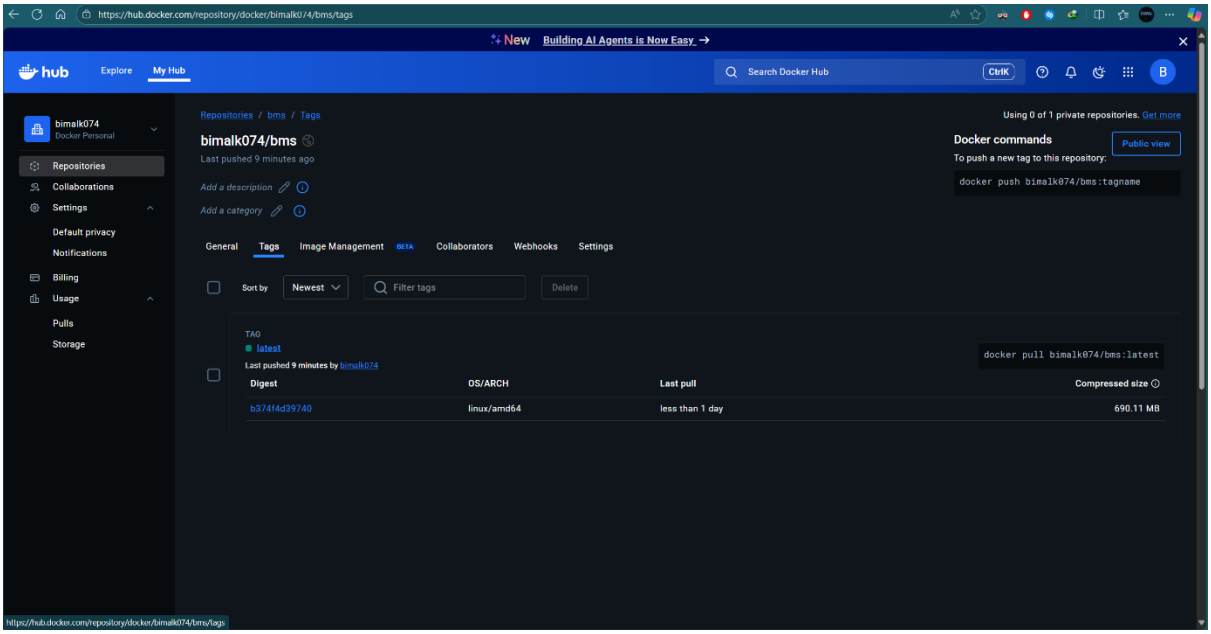


Figure 18: Docker image pushed to DockerHub via Jenkins pipeline

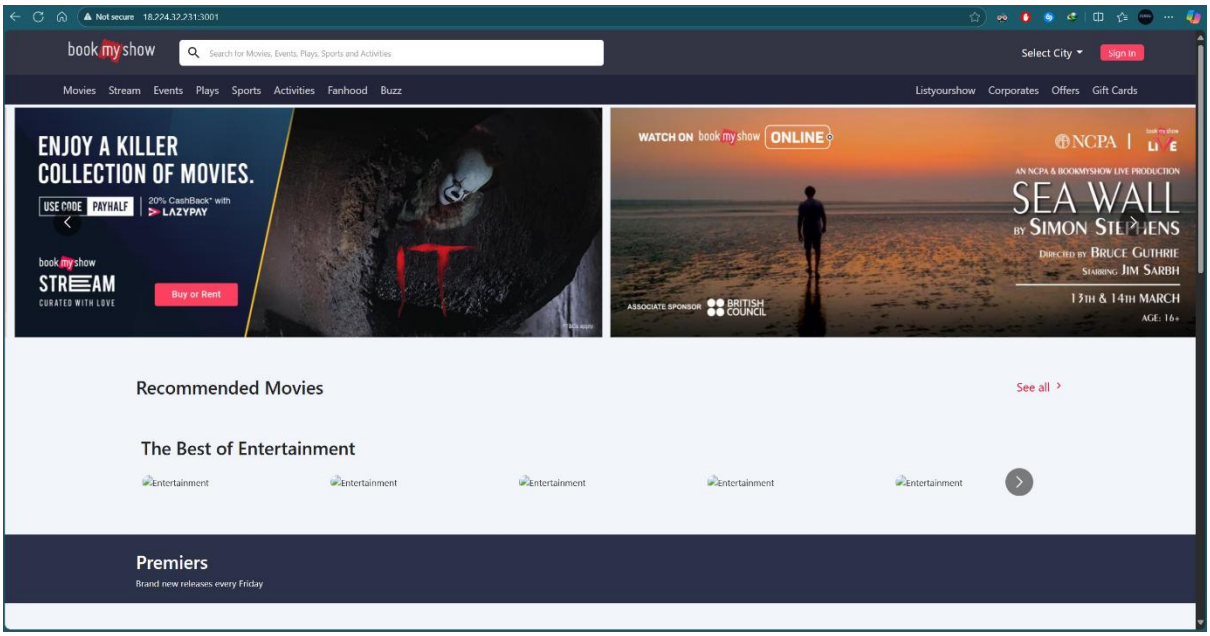


Figure 19: Application running in Docker container

Kubernetes Cluster Creation and Deployment (EKS)

- Installed eksctl, AWS CLI, and kubectl to manage the cluster.
- Created the cluster using eksctl create cluster and verified node status with *kubectl get nodes* (all nodes Ready).
- Cloned the GitHub repository in the EKS terminal.
- Created a namespace *bimal* and applied Kubernetes manifests (*deployment.yml* and *service.yml*).
- Exposed the application via LoadBalancer and validated deployment using *kubectl get pods* and *kubectl get svc*.

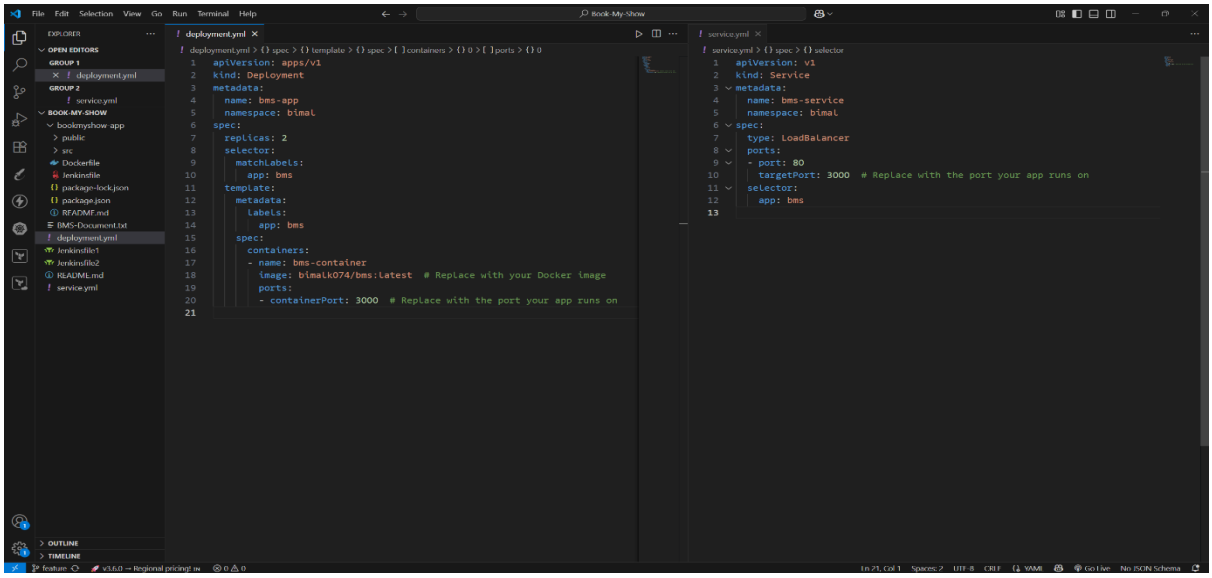


Figure 20: Deployment.yml and Service.yml files

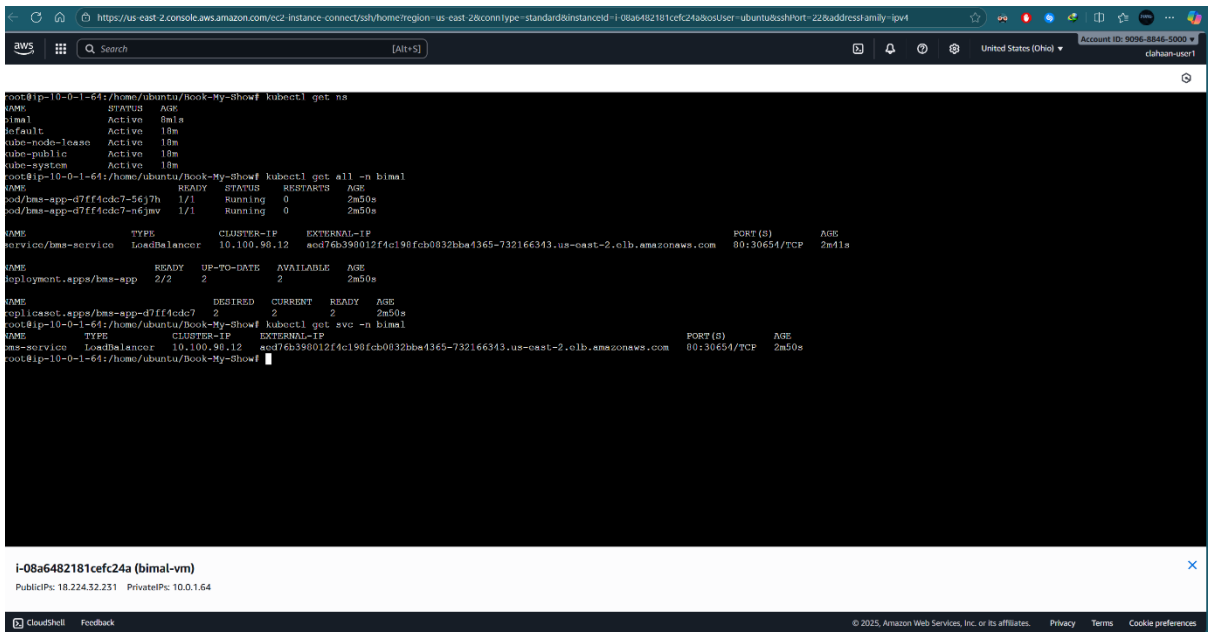


Figure 21: Kubernetes namespace, Pods, and Services running successfully

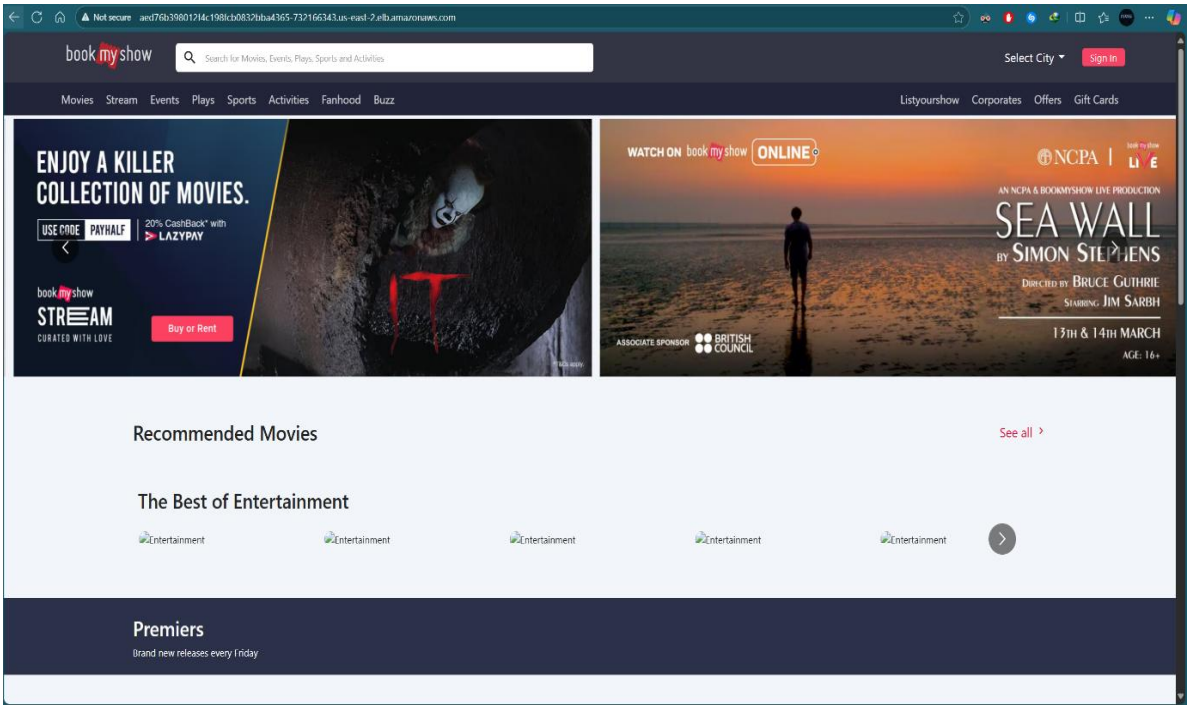


Figure 22: Application accessible through LoadBalancer

Monitoring & Observability

- Prometheus and Node Exporter were installed to collect metrics.
- Jenkins metrics were integrated into Prometheus, and Grafana was configured as a visualization tool.
- Dashboards for node health and Jenkins performance were created.

Prometheus and Grafana Setup:

- Helm was installed and the Prometheus & Grafana repository was added.
- Prometheus was deployed in the ‘monitoring’ namespace using *kube-prometheus-stack*.
- Grafana and Prometheus services were exposed via LoadBalancer.
- Grafana accessed at *http://<external-ip>:80*
- Prometheus accessed at *http://<external-ip>:9090*

Node Exporter Integration:

- Node Exporter was installed on cluster nodes to expose system-level metrics (CPU, memory, disk, network) to Prometheus.
- Prometheus was configured to scrape these metrics from *http://<node-ip>:9100/metrics*, enabling detailed monitoring of node health and performance.

Jenkins Integration:

- Jenkins was integrated with Prometheus via the Prometheus plugin.
- Build and job metrics were exposed and scraped by Prometheus to monitor Jenkins performance and pipeline health.
- Jenkins metrics was accessed at *http://<jenkins-server-ip>:8080/prometheus*.

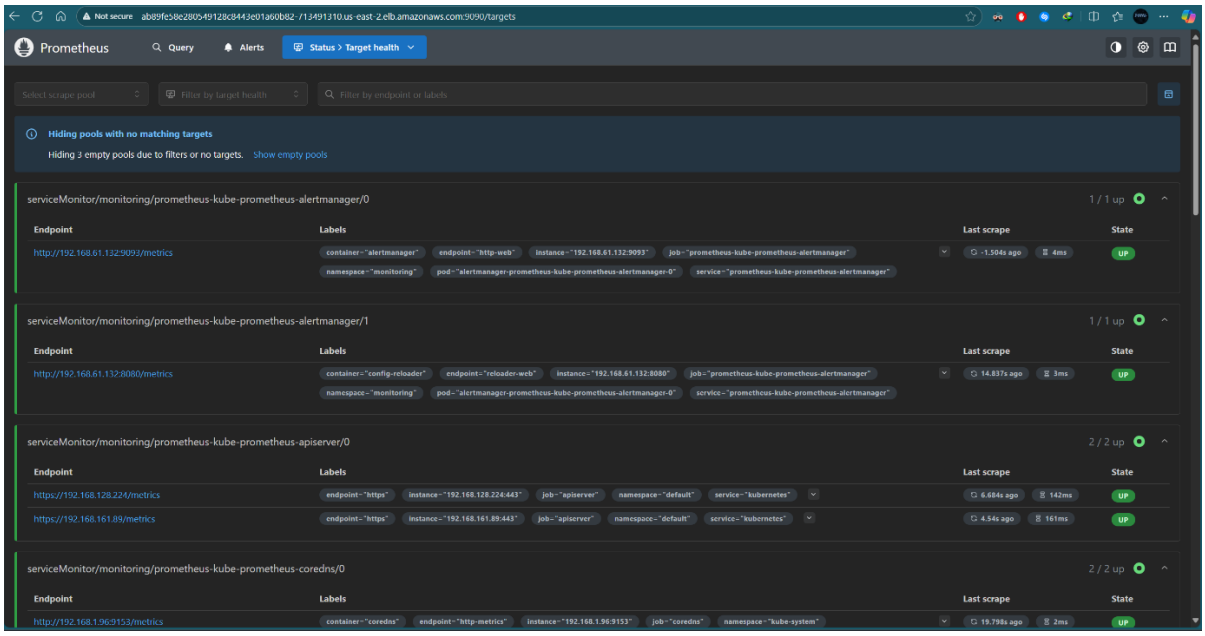


Figure 23: Prometheus installed and target health status

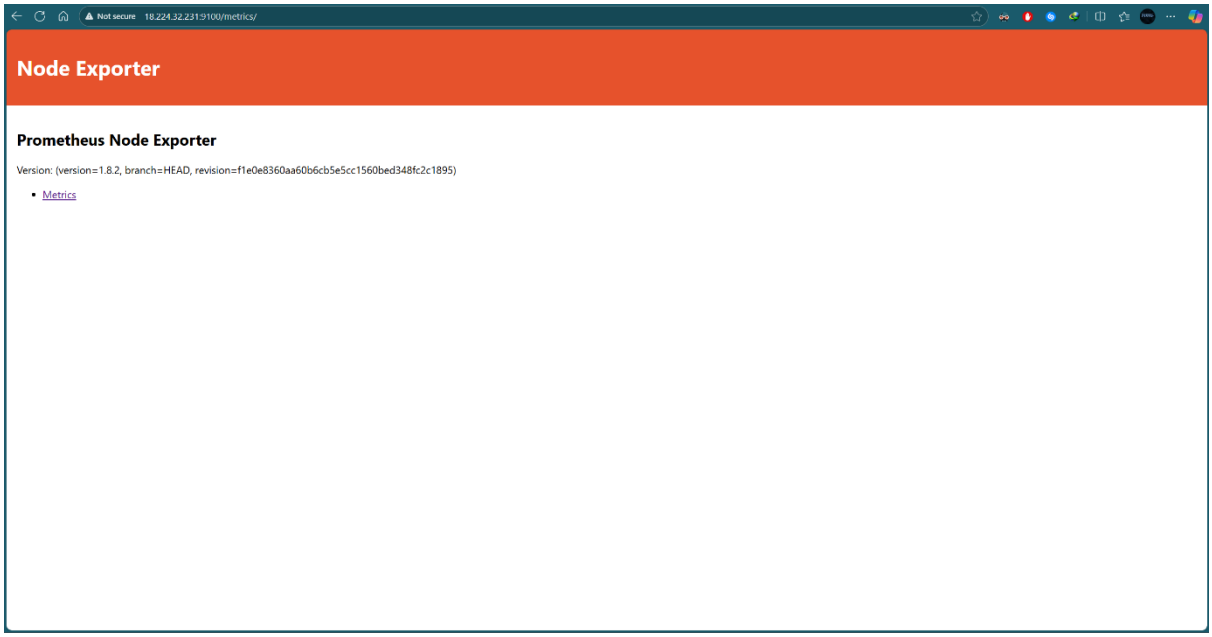


Figure 24: Node Exporter successfully installed and accessible

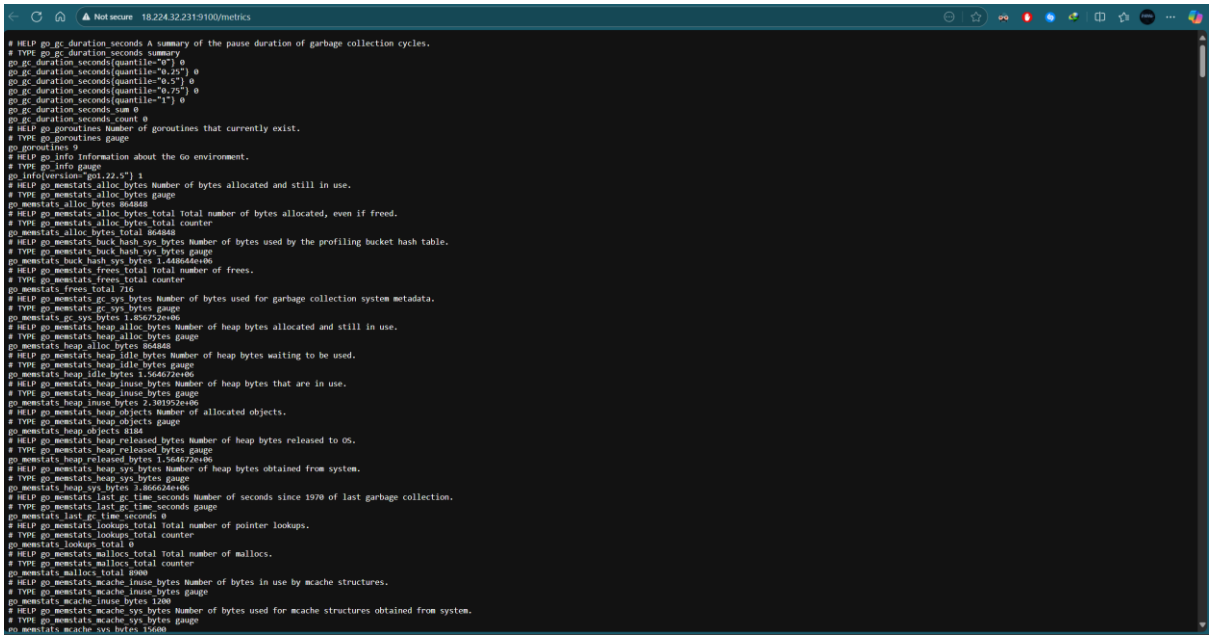


Figure 25: Node Exporter metrics endpoint showing system metrics

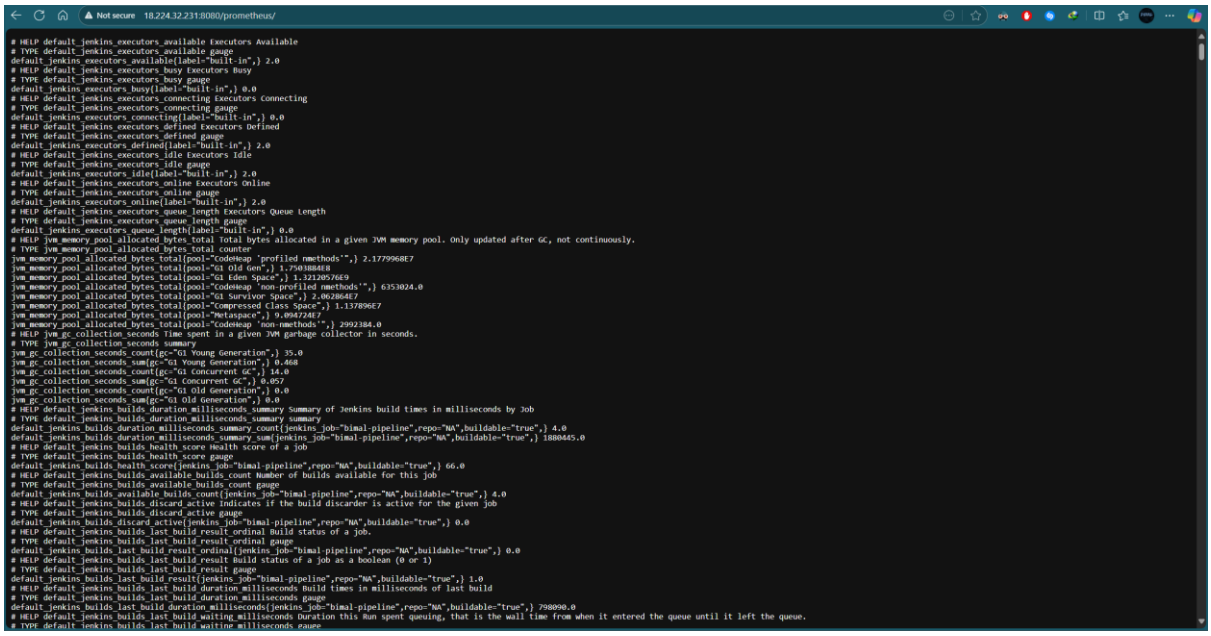


Figure 26: Jenkins metrics endpoint exposing build and system metrics

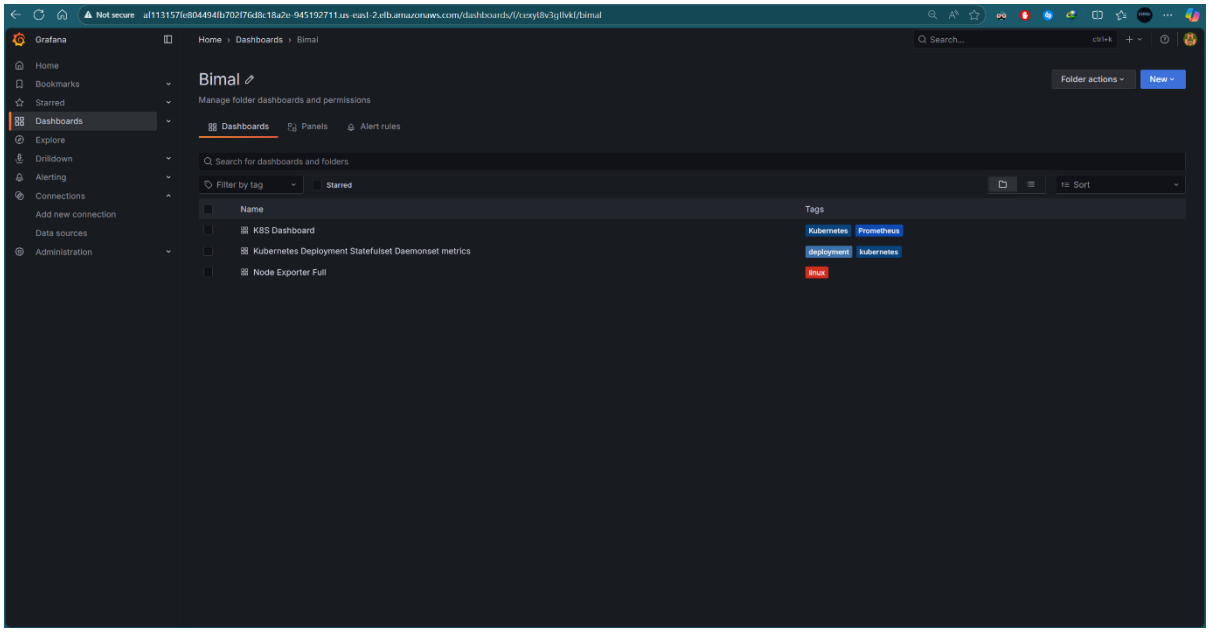


Figure 27: Grafana Dashboards

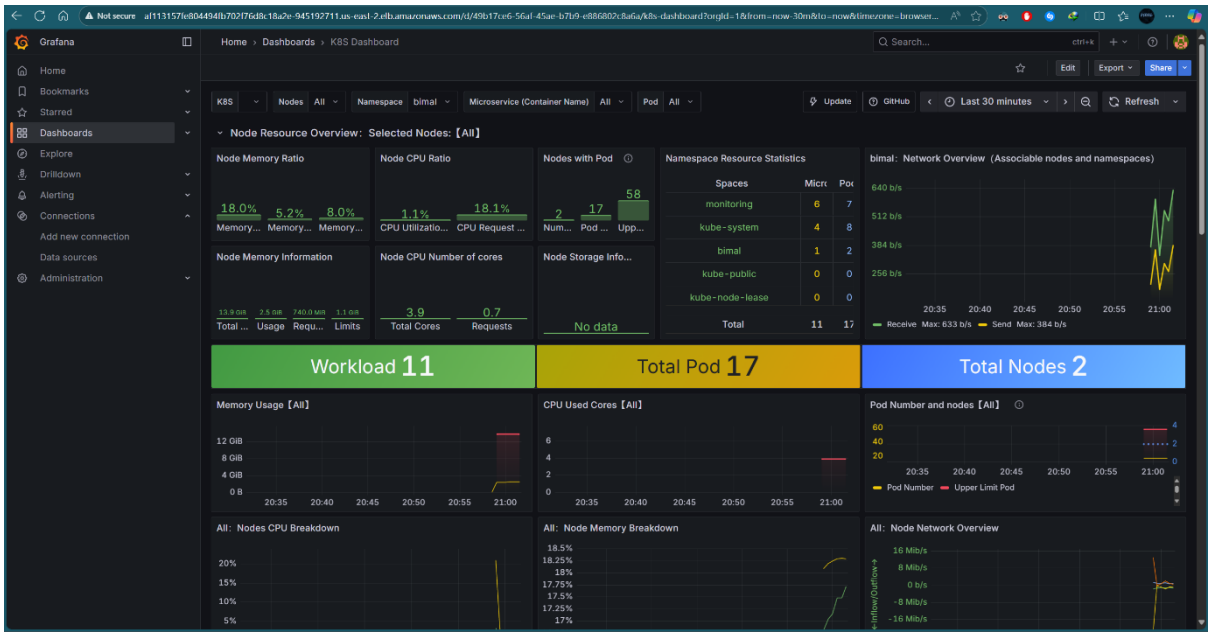


Figure 28: Kubernetes monitoring in Grafana

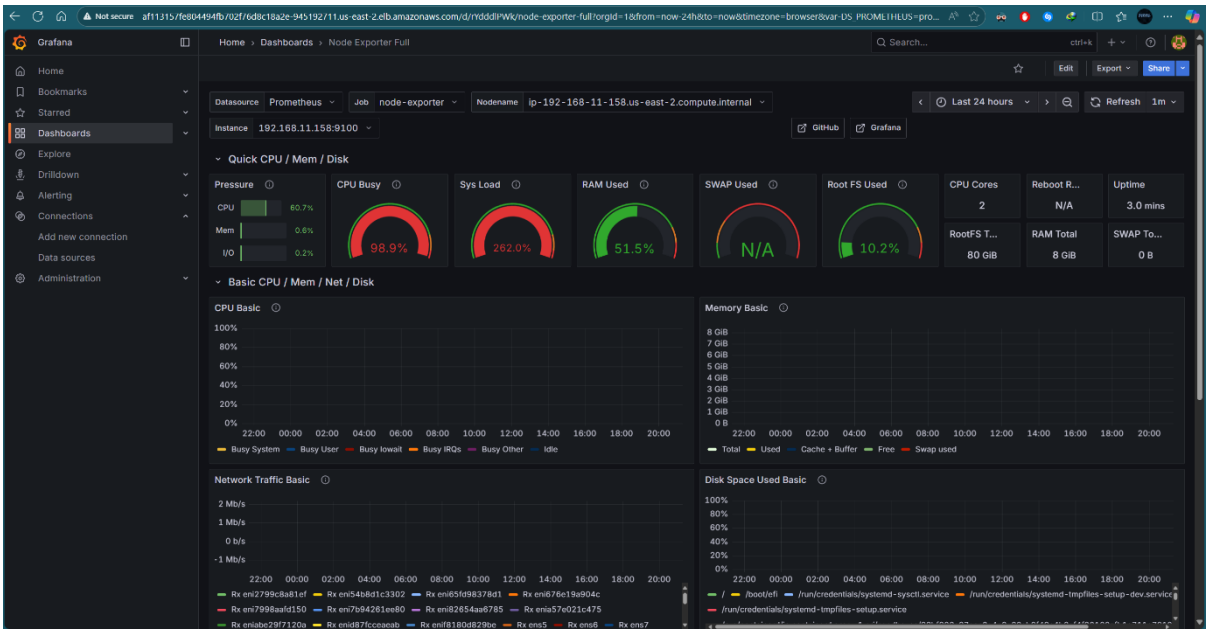


Figure 29: Node Exporter system metrics in Grafana

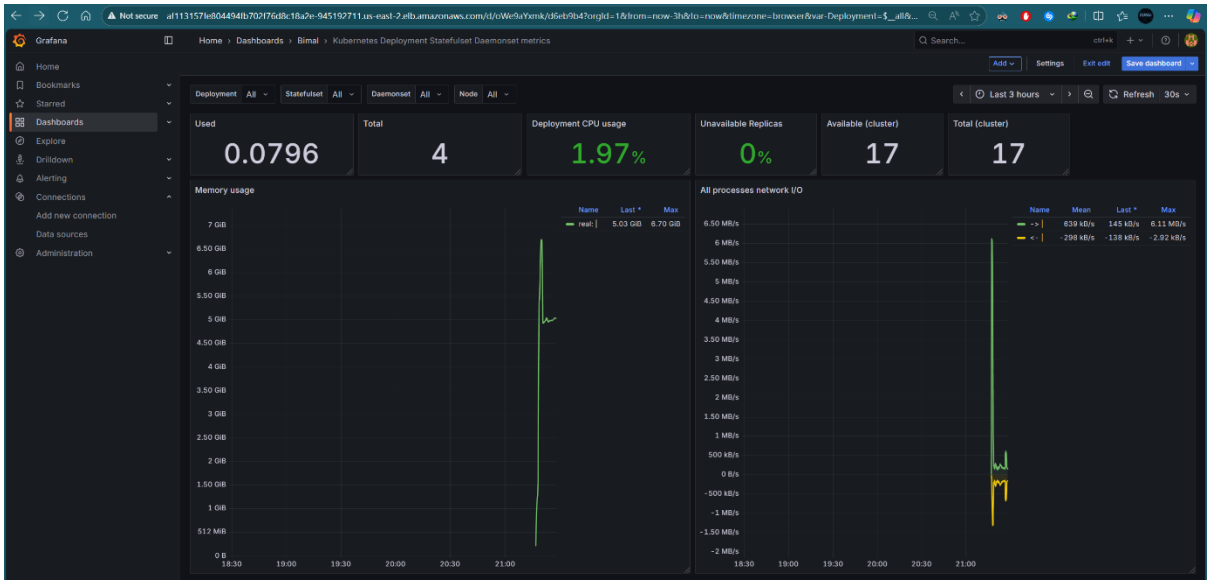


Figure 30: Kubernetes dashboard showing CPU, memory, network metrics